

ZEUS EVM

Relatório Executivo de Estado — 2026-06-22

Bot de arbitragem on-chain MEV em Base (com expansão multi-chain pronta). Este documento descreve o que o ZEUS faz, onde atua, como se comporta, e tudo o que está pronto pra ativar quando entrarmos em produção real.

Status geral	Pronto pra Phase 7 (mainnet com capital pequeno) após setup + 2 semanas DRY_RUN
Testes automatizados	Contratos 78/79 (unit) + fork verde · ~404 testes TS (execution-utils 336/336) · 0 falhas · typecheck 13/13
Workspaces validados	13/13 typecheck verde
Última atualização	22/06: OIE completa (toda inteligência → ledger+Grafana) + auditoria de fios soltos + Motor 2 vira executor (cross-DEX+triangular) · 15/06: OIE+ledger · 09/06: flashloan 0%
Pendência crítica	Capital inicial + deploy mainnet + 2 semanas DRY_RUN observando o ledger (execução do arb e EV gates são opt-in, default off)

1. O que é o ZEUS EVM

ZEUS EVM é um robô de software que opera 24/7 em redes blockchain do tipo EVM (Ethereum, Base, Arbitrum, Optimism, etc). Sua função é capturar oportunidades de lucro que aparecem por milissegundos quando outros usuários do mercado tomam decisões erradas, têm posições em risco, ou movimentam volume grande causando dislocação de preço.

■ **Em linguagem simples:** Imagine que existe uma feira gigante onde milhares de pessoas negociam o tempo todo. Algumas vezes alguém vende uma manga por R\$ 1 quando o preço justo é R\$ 2 — quem comprar primeiro e revender ganha R\$ 1. O ZEUS é o vendedor que está sempre olhando todas as barracas ao mesmo tempo e age no instante em que vê uma manga barata.

Três motores de lucro descorrelacionados

Motor	O que faz	Quando ganha mais
Motor 1: Liquidations	Liquida posições de empréstimo que ficaram com pouca garantia. Recebe um bônus do protocolo (5-15% do valor) por executar essa limpeza.	Em crashes / quedas de mercado
Motor 2: Cross-DEX Arbitrage	Compra um token barato numa exchange e vende caro em outra no mesmo instante.	Em mercados com volume alto
Motor 3: Backrun	Quando uma 'baleia' faz um swap grande que move o preço, o ZEUS opera logo depois pra restaurar o equilíbrio e lucrar com a dislocação.	Em mercados voláteis

■ **Em linguagem simples:** Os três motores são descorrelacionados — em qualquer cenário (crash, volume normal ou alta volatilidade) pelo menos um deles tende a estar ativo. É como ter três sócios que ganham em momentos diferentes: enquanto um descansa, o outro está trabalhando.

Como cada motor funciona, se posiciona e ganha dinheiro

Motor 1 — Liquidations (liquidações)

O que faz: em protocolos de empréstimo (Aave, Morpho, Moonwell...), quem pega dinheiro emprestado deixa uma garantia (colateral) como caução. Se o valor dessa garantia cai abaixo do limite, a posição fica 'no vermelho' e o protocolo abre pra qualquer um quitar parte da dívida e levar a garantia COM DESCONTO + um bônus (5-15%). O ZEUS é quem faz essa limpeza.

Como se posiciona: NÃO disputa as posições óbvias dos protocolos gigantes (Aave grande, onde mil robôs brigam por milissegundos — ali a gente perde). Foca nos protocolos de NICHOS e sub-servidos (Morpho, Moonwell, Seamless), onde há poucos liquidators competindo — vence quase sempre.

Como ganha: embolsa o bônus de liquidação. Sem capital próprio — um flashloan paga a dívida, o ZEUS recebe a garantia com desconto, vende, devolve o empréstimo e fica com a diferença.

■ **Em linguagem simples:** É como uma casa de penhor. Alguém empenhou um relógio de R\$ 2.000 por um empréstimo de R\$ 1.500 e não pagou. A loja deixa você quitar os R\$ 1.500 e levar o relógio. Você vende por R\$ 2.000 e lucra R\$ 500. O ZEUS é o cliente que sempre chega primeiro — e na loja certa, aquela vazia, sem fila.

Motor 2 — Cross-DEX Arbitrage (arbitragem entre exchanges)

O que faz: o mesmo token custa um pouco diferente em duas exchanges (pools) ao mesmo tempo. O ZEUS compra na mais barata e vende na mais cara — no mesmo instante, sem risco de preço.

Como se posiciona: NÃO disputa os pares óbvios (ETH/USDC é guerra de latência entre robôs — perdemos). Foca em pares sub-servidos (LSDs como cbETH/wstETH, stables fragmentadas) onde a

diferença de preço PERSISTE por mais tempo (não some em 1 bloco). O radar (MIS) construído hoje rankeia exatamente por PERSISTÊNCIA, e calcula o tamanho ótimo do empréstimo pra não estragar o próprio preço.

Como ganha: a diferença de preço entre os dois pools, menos a taxa e o slippage. Flashloan paga, faz a ida-e-volta (compra barato / vende caro), devolve e fica com o lucro.

■ **Em linguagem simples:** *É a feira de novo: a mesma manga custa R\$ 1,00 numa barraca e R\$ 1,10 na barraca do lado. Você compra na barata e vende na cara na mesma hora. MAS se comprar manga demais numa barraca só, o preço sobe na sua mão (slippage) — por isso o ZEUS calcula QUANTAS mangas dá pra mexer sem estragar o negócio. Esse cálculo é o que entregamos hoje.*

Motor 3 — Backrun (operar atrás da baleia)

O que faz: quando uma 'baleia' faz um swap GRANDE, ela empurra o preço do pool pra um lado (dislocação temporária). Logo ATRÁS dela, o ZEUS opera pra lucrar com esse desequilíbrio antes do preço voltar ao normal.

Como se posiciona: precisa VER a transação da baleia antes dela confirmar (isso exige acesso à 'mempool' premium). E opera LOGO ATRÁS (backrun) — não na frente da pessoa (não é o 'sandwich' predatório); é uma jogada mais limpa, só aproveitando o estrago que a baleia já causou.

Como ganha: o preço deslocado tende a voltar ao normal; quem opera primeiro nessa volta captura a diferença. Flashloan + operação atômica, igual aos outros.

■ **Em linguagem simples:** *Imagina que alguém compra TODAS as mangas de uma barraca de uma vez — o preço dispara por um instante. Você, logo atrás, vende as suas mangas pra essa barraca no preço inflado (ou compra na barraca do lado, que ainda está barata, e revende). Você lucra com o tranco que a compra gigante causou — sem ter empurrado ninguém.*

■ **Em linguagem simples:** *COMO GANHAMOS DINHEIRO NOS TRÊS (o fio comum): sempre via flashloan (empréstimo instantâneo que paga, opera e devolve na MESMA transação) — então NÃO precisamos de capital próprio parado. E tudo é ATÔMICO: se a operação não fechar com lucro, ela inteira é cancelada e só perdemos o gas (centavos). Nunca entra numa operação que pode dar prejuízo no meio. Dos lucros, 45% viram capital próprio reinvestido.*

Estado dos 3 motores hoje (2026-05-28)

Motor	Estado de código	O que falta pra ligar
Motor 1: Liquidations	PRONTO — edge decidido (Fase 4c: nicho sub-servido) + 5 protocolos implementados (Aave, Compound, Seamless, Morpho, Moonwell)	Deploy mainnet + capital + multisig + 2 semanas DRY_RUN (NÃO é estratégia — é execução/validação)
Motor 2: Cross-DEX Arb	RADAR PRONTO (hoje) — MIS varre/identifica/estima/ranqueia + sizing ótimo do empréstimo. Observação pura, não submete tx	RPC pago + dias de coleta de persistência + ponte radar→executor (passo futuro deliberado)
Motor 3: Backrun	MÁQUINA PRONTA — planner, bribe, bundling (Flashbots/Atlas/Blocknative), trackers. Feed de mempool é placeholder	Mempool premium (~US\$ 199/mês Alchemy) — único bloqueador; é ligar o cabo

■ **Em linguagem simples:** Os três estão ALINHADOS: um cérebro, um dataset (os colaterais sub-servidos), três braços descorrelacionados — zero retrabalho entre eles. Mas estão em FILA: cada um espera um destravamento diferente. O gargalo de hoje não é mais código — é decisão (capital/edge do Motor 1) + infra paga (RPC liga o Motor 2, mempool liga o Motor 3). Saímos da fase de construir e entramos na de ligar.

2. Mercados que o ZEUS ataca

2.1 Blockchains (chains)

Chain	Status atual	Observação
Base mainnet	Pronta pra ativar	Chain inicial — sequencer Coinbase
Base Sepolia (teste)	Contratos v8 deployados	Testes funcionando
Arbitrum Sepolia	Contratos v6 deployados	Pronta pra promover mainnet
Optimism Sepolia	Contratos v6 deployados	Pronta pra promover mainnet
Polygon	Code-ready (Motor 1)	Config + Deploy + liquidator ligados (2026-05-28) — falta só o deploy on-chain
Avalanche	Code-ready (Motor 1 + 2)	Motor 1 (29/05) + adapter Trader Joe LB pro Motor 2 (30/05). Falta deploy + fork test do TJ (requer RPC pago)

■ **Em linguagem simples:** Pense em cada blockchain como uma cidade diferente. O ZEUS já tem o veículo registrado em 3 cidades (Base, Arbitrum, Optimism) e a planta pronta pra registrar em mais 2 (Polygon, Avalanche). Quando o motor já roda no código, basta dar o endereço da nova cidade que ele opera lá também — sem reescrever nada.

2.2 Protocolos de empréstimo (lending)

Protocolo	Cobertura	O que é
Aave V3	Completa (todas chains)	Maior protocolo de empréstimo on-chain
Compound III	Completa (Base/Arb/Polygon)	Segundo maior, mecânica diferente
Morpho Blue	Contrato pronto, integração TS parcial	Mercados isolados, menos competição
Seamless (Base)	Planejado	Fork do Aave em Base, menos liquidators ativos
Moonwell (Base)	Planejado	Protocolo nicho — menos competição

■ **Em linguagem simples:** Cada protocolo é tipo um banco diferente onde pessoas pegam empréstimo. Quando o empréstimo fica 'no vermelho' (garantia caiu demais), qualquer um pode liquidar e ganhar um bônus. Bancos grandes (Aave) têm mais oportunidades mas mais competição. Bancos menores (Morpho, Moonwell) têm menos oportunidades mas você vence quase sempre.

2.3 Exchanges descentralizadas (DEXs) — onde o ZEUS faz os swaps

DEX	Status
Uniswap V3 (single-hop)	Funcionando
Uniswap V3 (multi-hop 2-3 pulos)	■ Implementado hoje
Aerodrome (Base nativo)	Funcionando

Velodrome (Optimism — fork Aerodrome)	Funcionando
BaseSwap	Endereços prontos, sem adapter
Trader Joe (Avalanche)	Necessário pra ativar Avalanche

3. O motor on-chain (smart contracts)

Toda operação do ZEUS executa via 4 contratos inteligentes deployados em cada chain. Eles são o 'pé no acelerador' — o resto do código off-chain só dá as ordens, mas é o contrato que efetivamente movimenta dinheiro.

Contrato	Responsabilidade
BribeManager	Paga gorjeta pro proposer do bloco quando precisamos passar à frente
ZeusLiquidator	Executa 3 tipos de liquidation: Aave, Compound e Morpho (Morpho é uma FUNÇÃO aqui dentro, não um contrato separado)
ZeusArbExecutor	Executa arbitragens cross-DEX, flashloan arbs e backrun
ZeusMoonwellLiquidator	Liquidation do Moonwell — contrato PRÓPRIO porque Moonwell é fork do Compound V2 (mecânica diferente) e não cabia junto (limite de tamanho)

■ **Em linguagem simples:** É como um cofre com 4 chaves: uma pra cada tipo de operação. Atenção: 'Morpho' NÃO é um contrato — é uma função dentro do ZeusLiquidator (junto de Aave e Compound). O 4º contrato é o do MOONWELL, que precisou ser separado porque o Ethereum limita o tamanho de cada contrato. Cada um passou por auditoria interna com 7 correções de bugs (B-1 a B-7).

■ **Em linguagem simples:** STATUS DE DEPLOY: os 3 primeiros já estão deployados em testnet (Base Sepolia). O ZeusMoonwellLiquidator (4º) foi adicionado ao script de deploy em 2026-05-29 e ainda NÃO foi deployado — entra junto no próximo deploy.

Mecanismos de segurança on-chain

Mecanismo	O que protege contra
MAX_TRADE_ETH	Tx tentando mover mais ETH do que o limite configurado
MAX_TRADE_PER_TOKEN	Tx tentando mover mais de algum token específico
KILL_SWITCH	Pausa total em emergência — dono mata o bot com 1 tx
approvedDexAdapters	Bot só fala com DEXs aprovadas — não chama contrato desconhecido
Atomic-only	Se qualquer passo falhar, a tx inteira reverte (não perde dinheiro pela metade)
Transient storage flag	B-7: evita ataque de re-entrada via coinbase pagamento

■ **Em linguagem simples:** Pense como os freios do carro: airbag, ABS, freio de emergência, freio de mão. Você espera nunca usar nenhum, mas em uma situação ruim, qualquer um deles te salva.

4. Como o ZEUS se comporta — passo a passo

A cada poucos segundos, o bot roda um ciclo completo de decisão. Antes de submeter qualquer transação real, a oportunidade passa por 8 portões de segurança em sequência — se qualquer um disser 'NÃO', a operação é cancelada antes de gastar gas.

Etapa	O que acontece
1. Descoberta	Bot consulta blockchain procurando posições liquidáveis ($HF < 1$)
2. Cálculo	Pra cada posição, simula o lucro ótimo (multi-collateral, partial amount, multi-hop)
3. Gate PnL Kill Switch	Bloqueia se já perdemos demais nas últimas 24h
4. Gate Cooldown	Bloqueia se tivemos N falhas seguidas (descanso forçado)
5. Gate Gas Reserve	Bloqueia se ETH na carteira está crítico
6. Gate AutoPause	Bloqueia se algum sinal de saúde está vermelho (reorg, RPC lento)
7. Gate Oracle Staleness ■	Bloqueia se preço Chainlink está velho demais
8. Gate Protocol Pause ■	Bloqueia se Aave/Compound estão pausados pelo governance
9. Gate Dedup	Bloqueia se a mesma posição já foi submetida agora
10. Simulação	Roda a tx em 'modo fantasma' (eth_call) — não gasta gas, vê se funciona
11. Stale Re-check	Última conferência: ainda vale a pena? HF mudou?
12. Dispatch	Submete a tx real pra blockchain
13. Pós-tx (PnL Reconciler)	Calcula lucro real vs esperado e arquiva pro aprendizado

■ **Em linguagem simples:** É como um piloto de Fórmula 1: antes da ultrapassagem ele faz uma checklist mental rápida — pneu OK, combustível OK, freios OK, distância OK, pista seca OK. Se algum item for NÃO, ele espera a próxima curva. Esse 'piloto' do ZEUS faz a checklist em milissegundos.

5. Funcionalidades implementadas nesta sessão (hoje)

Nesta sessão entregamos 6 melhorias do 'Grupo B' (gaps técnicos que faziam o bot perder oportunidades) + 4 itens finais da Fase 1 de infraestrutura. Tudo já está rodando, testado e integrado.

5.1 Grupo B — Gaps de captura fechados

Oracle Staleness Check

Antes de cada operação, o bot verifica se o preço usado (vindo do Chainlink) foi atualizado recentemente. Se está mais velho que o limite (ex: 1h), cancela a operação.

■ **Em linguagem simples:** *É como conferir a data de validade do leite antes de tomar. Se o preço está 'estragado', tomar uma decisão com ele dá errado.*

Pause Detection Upstream

O ZEUS confere se o protocolo (Aave ou Compound) está pausado pela governança antes de tentar liquidar. Sem isso, o bot submeteria tx que reverteria queimando gas.

■ **Em linguagem simples:** *É como olhar a placa 'FECHADO PARA REFORMA' antes de tentar entrar na loja. Tentar entrar mesmo assim só gasta tempo e energia à toa.*

Cache de Gas Price por Bloco

O bot agora reaproveita a consulta de preço de gas dentro do mesmo bloco em vez de perguntar pro RPC toda vez. Reduz drasticamente o número de chamadas externas.

■ **Em linguagem simples:** *É como anotar o preço do combustível ao chegar no posto em vez de perguntar pro frentista antes de cada litro. Economiza tempo e dinheiro.*

Multi-Collateral Evaluation

Antes, o bot só olhava o maior collateral + maior dívida de cada borrower (top-1 por peso). Agora avalia TODOS os pares possíveis (colateral X dívida) e escolhe o mais lucrativo.

■ **Em linguagem simples:** *Era como olhar só o item mais caro da prateleira de uma loja. Agora o ZEUS olha TODA a prateleira e escolhe o item com MELHOR margem — que nem sempre é o mais caro. Este foi o gap M-01 do audit: 26 de 28 oportunidades hoje não resolvem por causa disso.*

Partial Liquidation Optimization

O cálculo do tamanho ótimo da liquidation agora testa 16 amostras em escala logarítmica (vs 10 antes) + 2 fases de refinamento ($\pm 40\%$ depois $\pm 10\%$) pra travar no ponto ótimo.

■ **Em linguagem simples:** *É como mirar com uma luneta: antes você ajustava 1 vez e atirava. Agora ajusta de longe, ajusta de perto, e só então atira no centro exato.*

Multi-Path Swaps (2-3 pulos)

O bot agora pode trocar tokens não só direto ($A \rightarrow B$) mas também via intermediários ($A \rightarrow WETH \rightarrow B$ ou $A \rightarrow USDC \rightarrow B$). Em pools rasos, o caminho indireto às vezes dá MUITO mais saída do que o direto.

■ **Em linguagem simples:** *É como ir de uma cidade pequena pra outra: às vezes a estrada direta é ruim, mas se você passar pela capital primeiro, chega bem mais rápido. O ZEUS testa as duas opções e escolhe.*

5.2 Fase 1 — Finalizações entregues hoje

Item	O que faz
Failure Weekly Digest	Relatório semanal automático no Discord com TODAS as oportunidades perdidas, agrupadas por causa, protocolo, competidor que ganhou e hora do dia
InclusionCostBreakdown	Decompõe o custo de inclusão em 4 componentes (base fee queimado, gorjeta pro proposer, custo L1, bribe coinbase) — pra calibrar onde cortar
PnL Weekly Deep Dive	Relatório semanal com análise por protocolo, venue, par, hora — tipo ' fechamento contábil' do bot
Protocol Affinity Tracker	Identifica em qual protocolo cada competidor é especialista (95% Aave-only? Diversificado?)
Multi-Signal Classifier	Combina 5 sinais (gás, atividade, builder, etc) pra classificar competidores com confiança calibrada
Cooccurrence Analyzer	Detecta quando vários endereços operam juntos sempre — indica sybil (mesmo dono com várias carteiras) ou pares sandwich
Grafana Dashboards	2 dashboards prontos: Operations & Health + Performance & Latency (23 painéis totais)

5.3 Motor 2 — Radar de ineficiência (MIS) construído nesta sessão

Entregamos o MIS (Market Inefficiency Scanner) — o radar do Motor 2 (arbitragem cross-DEX). Ele roda em OBSERVAÇÃO PURA: lê o estado dos pools on-chain, mede divergências de preço, estima a economia real de um flashloan e ranqueia por PERSISTÊNCIA. Não toca em capital, não submete transação. É um radar — e radar não atira.

Peça	O que faz
Varredura em multicall	Lê o estado de TODOS os pools de TODOS os pares em 1 ida-e-volta ao RPC (em vez de uma chamada por pool). Essencial pra escalar pra dezenas de pares.
Derivação de tokens on-chain	Em vez de digitar tokens à mão (e errar), o MIS lê os colaterais direto dos protocolos de lending (Aave/forks, Moonwell, Morpho) — endereços garantidos pela fonte — e auto-popula os pares a monitorar. Motor 1 e Motor 2 passam a compartilhar o mesmo conjunto de tokens.
Estimador de flash (quoter)	Quando acha divergência, traduz em números REAIS via quoter on-chain: par, hora, valor do empréstimo, valor de devolução à Aave (+0,05%), custo de gas, lucro bruto/líquido em \$ e %.
Gate de profundidade	Roda o round-trip do empréstimo no quoter; se o pool é raso (slippage devora o trade), o par é marcado RASO e EXCLUÍDO do ranking. Tira do mapa as 'oportunidades' que são só slippage disfarçado.
Sizing do empréstimo (optimizeFlashLoan)	Varre tamanhos de empréstimo (\$1k→\$250k) via quoter e acha o PICO de lucro (maior antes do slippage matar o edge) + o TETO VIÁVEL (maior empréstimo ainda lucrativo). Para cedo quando passa do pico (poupa RPC). Saber quanto dá pra pegar sem inviabilizar é fundamental.
Persistência (liga/desliga)	O histórico é salvo em disco e recarregado ao reiniciar — a persistência (sinal-chave) acumula dia após dia mesmo sem rodar 24/7.

■ **Em linguagem simples:** O gate de profundidade foi a lição da sessão: divergências de spot que pareciam ótimas (ex: 97 bps) dão PREJUÍZO de ~99% num flash de \$10k, porque os pools daquele par são rasos. Só o quoter revela isso. Por isso o radar agora separa 'divergência bonita' de 'oportunidade que aguenta nosso tamanho'.

■ **Em linguagem simples:** IMPORTANTE — o MIS NÃO está ligado ao executor de flashloan, e isso é de propósito. Ele varre, identifica, estima e ranqueia — e PARA no log. Ligar a execução é um passo futuro deliberado (ver seção 8), que só faz sentido depois de dias de persistência coletada + as peças de segurança do arb prontas. Detalhe na seção 8.

5.4 Validação on-chain — endereços, ABIs e flashloan (2026-05-29)

Confirmamos contra a mainnet REAL (via eth_call read-only) que os endereços e ABIs que o ZEUS usa batem com o que está deployado, e que o premium do flashloan da Aave é o que o código assume (0.05%). Isso vale pras 3 chains de expansão do Motor 1 (Base/Polygon/Avalanche) + o adapter Trader Joe do Motor 2.

Chain	Aave (premium / reserves / provider)	DEXs (resolve + ABI)
Base	0.05% ✓ · 15 reserves · provider ✓	UniV3 WETH/USDC ✓
Polygon	0.05% ✓ · 21 reserves · provider ✓	UniV3 WETH/USDC ✓
Avalanche	0.05% ✓ · 18 reserves · provider ✓	UniV3 WETH.e/USDC ✓ · Trader Joe LB: 4 pairs, spot WAVAX≈\$8.82 ✓

■ **Em linguagem simples:** O spot do Trader Joe (AMM por bins) saiu correto (\$8.82/AVAX, faixa sã) lendo direto do getSwapOut on-chain — isso valida a orientação/decimais do adapter LB SEM precisar de fork. A lógica dos contratos (4 motores) está coberta por 67/68 unit tests verdes.

Fork test de EXECUÇÃO do flashloan: dRPC free bloqueia, Alchemy free RESOLVE

O fork test executa o flashloan ponta-a-ponta contra o estado REAL da mainnet. O dRPC FREE BLOQUEIA: ao buscar storage on-chain (eth_getStorageAt) retorna HTTP 408 'upgrade to paid'. MAS trocando pro Alchemy (free tier), funciona — o Alchemy serve archive/storage no grátis. Rodamos a suíte de fork completa contra a Base mainnet via Alchemy: 31/31 PASSAM.

Suíte (fork Base mainnet via Alchemy free)	Resultado
ZeusLiquidator (Motor 1)	7/7 — flashloan→Aave, guards, kill switch, endereços reais
ZeusArbExecutor (Motor 2/3)	9/9 — inclui quebrar preço e LUCRAR (arb) + flashloan + backrun
BribeManager (gorjeta MEV)	15/15 — coinbase, anti-sandwich (H-01), refund, transient flag

■ **Em linguagem simples:** Conclusão da infra de RPC: o ALCHEMY FREE já basta pra (a) confirmar endereços/ABIs/premium, (b) rodar a lógica (unit), e (c) SIMULAR a execução real do flashloan contra a mainnet (fork test) — o teste definitivo de 'funciona de verdade'. O tier PAGO só é necessário pro Motor 3 (mempool ao vivo), não pros testes. O dRPC free serve pra leitura/discovery; pra fork test, Alchemy.

5.5 Prova de LUCRO ponta-a-ponta dos 3 motores (fork Base mainnet)

O teste definitivo: cada motor executa o flashloan ponta-a-ponta contra o estado REAL da Base e fecha LUCRO. A técnica é 'quebrar o preço' no fork (um sandbox descartável, não toca a mainnet real) pra criar a condição que o motor explora, e validar que a lógica empresta, opera, devolve o flashloan + premium (0.05%) e sobra lucro. Os 3 passaram.

Motor	Cenário criado no fork	Resultado (lucro líquido)
Motor 1 — Liquidation	Borrower fica underwater (oracle drop → HF 0.74), liquida 50% via flashloan	+US\$ 6.157 (realista: bônus – premium – swap)
Motor 2 — Cross-DEX Arb	Whale dump cria gap → flashloan compra barato (pool 3000) e vende caro (pool 500)	+US\$ 371k* (prova a mecânica)
Motor 3 — Backrun	Mesma dislocação + paga bribe ao block.coinbase	+US\$ 334k* líquido pós-bribe

■ **Em linguagem simples:** * Os valores do Motor 2/3 estão INFLADOS de propósito: dumpamos 800 WETH pra criar um gap gigante e provar que a mecânica do flashloan fecha (empresta → 2 swaps → devolve emprestimo+premium → lucra → transfere). O sizing REALISTA por oportunidade é o que o optimizeFlashLoan/MIS calcula off-chain. O lucro do Motor 1 é realista (10 WETH colateral, ~50% liquidado, bônus ~7%). Suíte de fork completa: 34/34 verdes (31 de wiring/segurança + 3 de lucro).

5.6 O que o fork test prova (e o que NÃO prova) — leia com atenção

■ *Em linguagem simples:* Esse lucro NÃO era dinheiro real esperando na mainnet. O cenário foi FABRICADO dentro do fork (sandbox descartável): nós criamos o borrower do zero e forçamos o preço cair via mock do oracle — nada disso aconteceu na Base real. Logo: se o bot estivesse ligado AGORA, NÃO teríamos feito esses \$6.157. O teste prova que a LÓGICA captura o lucro corretamente QUANDO a oportunidade existe — não que ela existia.

O teste PROVA ✓	O teste NÃO prova ✗
A matemática do lucro (bônus – premium – swap) está certa	Que existia essa oportunidade na mainnet agora
A execução do flashloan funciona ponta-a-ponta	Que o preço caiu de verdade (foi mock)
O bot captura o lucro quando a condição existe	Que ganharíamos a corrida contra outros liquidadores

■ *Em linguagem simples:* Pra virar dinheiro REAL precisa de TUDO junto: (1) bot deployado e ligado na mainnet; (2) um borrower real ficando underwater — acontece em QUEDA de mercado, não em mercado calmo; (3) a discovery achar antes dos concorrentes; (4) a tx ganhar a corrida (gas/MEV) e entrar no bloco. Lucro real HOJE = US\$ 0 (não está deployado + cenário fabricado). O que temos é o 'carro provado na pista de testes' — falta ligar na rua, e a rua só paga quando o mercado se mexe e chegamos primeiro.

Como rodar: `pnpm contracts:test:fork` (usa Alchemy automático via `ALCHEMY_API_KEY` do `.env`). Confirmação read-only de endereços/ABIs: `apps/mis-scanner/scripts/confirmOnchain.ts`.

5.7 Flashloan multi-fonte 0% — Morpho + Balancer (2026-06-09)

Até agora, TODA operação do ZEUS pegava o empréstimo-relâmpago (flashloan) na Aave V3, que cobra 0,05% de taxa SEMPRE — inclusive nas liquidations de Morpho, onde o próprio Morpho emprestaria de graça. Inspirado na documentação de flashloans da Enso, tornamos a FONTE do empréstimo selecionável: o ZEUS agora escolhe a mais barata COM liquidez — prioridade Morpho (0%) → Balancer (0%) → Aave (0,05%, sempre disponível como rede de segurança). Vale pros 3 contratos (Liquidator, ArbExecutor, Moonwell).

■ **Em linguagem simples:** É como pegar dinheiro emprestado pra quitar uma dívida: você estava indo no banco que cobra juros (Aave 0,05%) toda vez — mesmo quando o banco da esquina te emprestaria DE GRAÇA pra isso. Agora o ZEUS olha primeiro o de graça (Morpho/Balancer) e só usa o que cobra juros se os de graça não tiverem o valor na hora.

Por que importa em dinheiro: num flash de US\$ 50k, 0,05% = US\$ 25 jogados fora por operação. Pior: numa liquidation disputada (vários bots brigando), esses 0,05% são exatamente a diferença entre fechar no lucro e reverter. Eliminar a taxa aumenta o win rate marginal.

Fonte	Taxa	Liquidez disponível
Morpho Blue	0%	Saldo inteiro do token no singleton (todos os mercados + colateral somados) — funda em tokens majoritários
Balancer V2 Vault	0%	Saldo do token no Vault — funda em WETH/USDC
Aave V3 (fallback)	0,05%	Universal — presente em quase todo token; rede de segurança

Segurança: o vetor de hijack do Balancer (e como foi fechado)

O callback do Balancer tem um risco real: qualquer um pode chamar o Vault mandando ELE invocar nosso contrato com dados maliciosos (o 'msg.sender' seria o Vault legítimo, então passaria a checagem ingênua). A defesa é uma 'flag de bilhete' em transient storage: o nosso entryptpoint marca 'EU iniciei este flash' ANTES de chamar a fonte, e o callback só aceita se o bilhete bater. Sem o bilhete, reverte — fechando o vetor. Os cores de liquidation/arb auditados ficaram INTACTOS (a taxa já era um parâmetro deles → passar 0 não exigiu reescrevê-los).

■ **Em linguagem simples:** É como a catraca de um show: o segurança (Vault) pode te levar até a porta, mas só entra quem tem a pulseira que NÓS colocamos na entrada. Um impostor que o segurança trouxe sem pulseira é barrado na catraca.

Validação on-chain (fork Base mainnet via Alchemy)

Rodamos o fluxo ponta-a-ponta contra os contratos REAIS da Base e lemos o trace de execução. Confirmado: o Morpho aceitou nosso flashLoan de 100 USDC e transferiu SEM taxa; nosso callback onMorphoFlashLoan disparou; a flag anti-hijack foi consumida corretamente; e o fluxo seguiu até o liquidationCall (que reverte só porque o usuário de teste é saudável — exatamente onde deveria). No Balancer, o trace mostrou feeAmounts=[0] — 0% confirmado on-chain.

O que foi validado	Resultado
Round-trip Morpho 0% (fork Base real)	flashLoan aceito → transfer s/ premium → callback → dispatch → revert no HF (correto)
Round-trip Balancer 0% (fork Base real)	flashLoan aceito → feeAmounts=[0] → receiveFlashLoan → dispatch → revert no HF (correto)
Anti-hijack + caller-spoof (unit)	Reverte InvalidCaller mesmo fingindo ser o Vault sem a flag

Suíte completa (unit + fork)	114 testes verdes · 0 falhas · 13/13 typecheck
------------------------------	--

■ **Em linguagem simples:** O revert acontecer DENTRO do fluxo (no liquidationCall, porque o usuário de teste é saudável) e NÃO na hora de pegar o empréstimo é a prova de que a chamada do flashloan 0% funciona de verdade — o Morpho/Balancer emprestou, nosso contrato recebeu e o caminho seguiu. Faltou só um devedor de verdade no vermelho pra fechar lucro.

■ **Em linguagem simples:** RESSALVA HONESTA: isso mexeu em código que já passou por auditoria interna — então NÃO vai pra mainnet sem auditoria interna nova + 2 semanas de DRY_RUN, igual à regra de todo o resto. E o motor de arbitragem (ArbExecutor) ganhou a CAPACIDADE on-chain, mas o seletor 0% ainda não foi ligado a ele (segue em fase radar — usa Aave por padrão lá). O ganho garantido hoje é no Motor 1.

5.8 OIE — motor de inteligência de oportunidade (2026-06-15)

O OIE dá ao ZEUS um 'senso de prioridade': em vez de só passar/não-passar um gate binário, ele PONTUA cada oportunidade por valor esperado ($EV = \text{probabilidade de ganhar} \times \text{lucro líquido}$) e escolhe a melhor quando várias disputam o mesmo bloco. Foi a resposta ao bloqueador #1 (estratégia com edge) — e veio com um achado de pesquisa que mudou o foco do motor de liquidação.

O achado decisivo: OEV está fechando a liquidação na Base

A pesquisa de mercado (docs/refs/competitive-landscape.md) mostrou que os protocolos grandes passaram a CAPTURAR pra si o lucro da própria liquidação (OEV — Oracle Extractable Value), via Chainlink SVR e 'MEV tax'. Sobra quase nada pro liquidador externo — EXCETO no Morpho Blue, que segue aberto e entrega o bônus inteiro (~4,9%) a quem liquida.

Protocolo (Base)	OEV capturado pelo protocolo	Edge pro ZEUS
Morpho Blue	0% (aberto)	■ EDGE REAL — foco do bot
Aave V3 (ativos principais)	~85% (Chainlink SVR)	■■ quase nulo
Compound III	~85% (SVR/Atlas)	■■ quase nulo
Moonwell	~99% (MEV tax on-chain)	■ praticamente nulo

■ *Em linguagem simples:* Pensa numa casa de penhor que paga uma gorjeta pra quem traz cliente. Antes, o ZEUS era esse trazedor e embolsava a gorjeta. Agora a maioria das casas (Aave/Compound/Moonwell) mudou a regra: a PRÓPRIA casa fica com a gorjeta (isso é o OEV). Só uma casa ainda paga a gorjeta cheia pra quem traz: o Morpho. Então o ZEUS foca onde ainda sobra dinheiro pra ele.

Como o OIE aplica isso (com segurança)

O liquidador agora calcula o lucro REALISTA pós-OEV ($\text{nominal} \times (1 - \text{recapture})$) e pontua cada oportunidade. Há um 'gate' opcional (MIN_OPPORTUNITY_EV_USD) que, quando ligado, descarta as liquidações antieconômicas ANTES de gastar gas — fazendo o bot focar em Morpho naturalmente. O backrun ganhou um gate parecido, ciente de 'guerra de gas' (corta corridas que perderia). Tudo descorrelaciona com o flashloan 0% da 5.7: Morpho é o melhor protocolo PRA liquidar E a fonte de empréstimo mais barata.

■ *Em linguagem simples:* IMPORTANTE — o gate vem DESLIGADO por padrão. Hoje ele só LOGA o score de cada oportunidade (observabilidade); o comportamento do bot não muda até você ligar com uma variável de ambiente. Os números de OEV (85%/99%) são defaults de pesquisa — a ideia é observar os logs no DRY_RUN e calibrar com dado real antes de ligar o corte. Validação: scoring 33/33 testes verdes.

5.9 Ledger de observação DRY_RUN + varredura dinâmica + deploy Fly.io (2026-06-15)

Pra provar (e não só assumir) onde está o edge, o detector de arbitragem e o MIS scanner agora GRAVAM tudo o que observam num banco local (DuckDB) — antes só logavam e esqueciam. Cada oportunidade vista vira uma linha no 'diário' do bot, ranqueada por PERSISTÊNCIA (quantas horas distintas aquele par apareceu — o sinal-chave de edge real, não sorte de um bloco).

Peça	O que faz
Ledger de observação (DuckDB)	Detector grava 'arb_observed', MIS grava 'mis_observed'. Cada motor tem seu arquivo (DuckDB é single-writer); a unificação cross-motor é na consulta (ATTACH).
Ranking por persistência	Responde empiricamente 'quais pares pagam de verdade' = frequência + lucro médio + horas ativas distintas.
Varredura dinâmica (detector)	Detector passou a consumir getTargetPairsForChain: pares curados + auto-targets do scraper. Rodar o scraper amplia a cobertura SEM mexer em código.
Deploy Fly.io persistente	Dockerfile + 3 configs (detector/MIS/liquidator) com VOLUME persistente — sem ele o diário zera a cada restart. Detector com KILL_SWITCH=true (observação pura, nunca submete).

■ **Em linguagem simples:** É o bot virando um pesquisador disciplinado: ele anota TODA oportunidade que vê — inclusive as que não executa — num caderninho que sobrevive a desligamentos. Em semanas, o caderninho responde com número, não achismo: 'esses pares aqui têm edge de verdade; esses só pareciam'. É exatamente o que falta pra decidir, com dado, o que vai pra mainnet.

■ **Em linguagem simples:** Por que isso importa AGORA: o caminho pro primeiro lucro real sempre teve '2 semanas de DRY_RUN' como porteira. Esta entrega é a INFRA que faz esse DRY_RUN medir de verdade (rodando 24/7 na Fly.io, gravando no volume) — em vez de observar e esquecer. Validação: ledger 6/6 testes verdes (Windows e Linux), suíte execution-utils 295/295, typecheck 13/13.

5.10 OIE completa — toda a inteligência: lida → gravada → Grafana (2026-06-22)

A camada de inteligência foi FECHADA: todos os sinais que antes viviam soltos (em RAM ou JSON, fora do banco central) agora caem no ledger DuckDB E aparecem numa métrica/painel Grafana. O bot deixou de 'ver e esquecer' — passou a registrar e enxergar tudo num lugar só.

Sinal capturado	O que é
Market-bribe	Quanto os competidores estão pagando de gorjeta — e isso vira o piso do nosso próprio bribe (não pagar de menos e perder, nem de mais e queimar lucro).
Perfis de competidor + sybil	Quem disputa contra nós, hábitos de gás, e quando várias carteiras são o mesmo dono (sybil).
Reconciliação de PnL	Lucro esperado vs. real, com a diferença decomposta — pra calibrar.
Falhas categorizadas + post-mortem	Por que perdemos e quem ganhou a corrida (com posição no bloco).
Thresholds adaptativos (Etapa C)	O bot recalcula sozinho os limites de EV a partir do histórico — opt-in (default off).

■ **Em linguagem simples:** Imagina um detetive que antes anotava pistas em guardanapos espalhados e perdia metade. Agora ele tem UM caderno central + um quadro na parede (Grafana) onde tudo aparece organizado: quem são os concorrentes, quanto pagam, onde perdemos, quanto lucramos de verdade. Em semanas isso vira calibração com número, não achismo. O motor 3 (backrun), que era invisível, também passou a publicar suas métricas.

5.11 Fios soltos — auditoria honesta + remediação (2026-06-22)

Uma auditoria global sem dourar a pílula: na prática, a tese dos '3 motores' se reduzia a 1 motor parcialmente estrangulado. O documento mapeou cada ponta solta por severidade e a maioria foi corrigida em fases (cada uma com teste verde). Honestidade > otimismo.

Fio solto	Correção
Liquidator sem fallback de RPC	dRPC → Alchemy com failover automático do viem (no-op se só há 1 RPC).
Aave/Seamless travados na chave do TheGraph	Discovery on-chain SEMPRE; TheGraph vira só acelerador opcional.
Seletor flashloan 0% não ligado no arb	Portado pro motor de arb (Morpho/Balancer 0% > Aave 0,05%).
Qualidade de dado contaminando o dataset	Gás nunca mais vira \$0 (lucro inflado); config validada (sem loop travado); priority fee real; endereço Moonwell checado no boot.
Classes de inteligência 'órfãs'	PnlAggregator, drift, post-mortem ligados no liquidator — dormentes em DRY_RUN, prontas pra MAIN.

■ **Em linguagem simples:** É como passar um pente-fino no carro antes da viagem: nenhum parafuso solto. Cada item era uma falha silenciosa que só apareceria na pior hora (RPC caindo, gás calculado errado, endereço com typo). Agora estão apertados. Duas pontas ficaram deferidas de PROPÓSITO porque dependem de infra paga, não de código: o feed de mempool do motor 3 e um volume de deploy.

5.12 Motor de Arb — o Motor 2 ganha mãos + visão triangular (2026-06-22)

O Motor 2 (varredura de ineficiências) era só um RADAR: via oportunidades mas não fazia nada. Agora ele virou um MOTOR DE EXECUÇÃO cross-DEX, ganhou toda a camada de inteligência dos outros motores, e aprendeu a enxergar arbitragem TRIANGULAR (A→B→C→A, não só par a par). O Motor 3 fechou as 2 últimas pontas. Os 3 motores ficam no mesmo nível de software.

As travas de segurança do novo executor (todas presentes)

Trava	Estado
Execução DESLIGADA por default	Sem env deliberada, o Motor 2 continua só observando (radar). O merge NÃO liga execução.
Circuit breakers	MAX_TRADE_ETH + lucro mínimo + slippage máx, validados no boot.
Chave exclusiva	EXECUTOR_PRIVATE_KEY própria do motor — regra inviolável, sem reuso.
Simula + EV gate antes de disparar	eth_call obrigatório + filtro de EV antes de qualquer gasto; re-cota fresco no instante do disparo.
Flashloan-only / atômico	Falha = só gás, nunca capital próprio.

■ **Em linguagem simples:** O Motor 2 tinha olhos mas não tinha mãos. Agora ganhou mãos — mas com TODAS as travas: só trabalha se você ligar de propósito, nunca arrisca dinheiro próprio (empréstimo-relâmpago), e confere tudo num ensaio antes de agir. E ficou mais esperto: além de comparar 2 feiras, vê ciclos de 3 (compra manga, troca por banana, troca por dinheiro — e sobra). A detecção triangular já funciona; a execução dela é o próximo passo.

■ **Em linguagem simples:** RESSALVA HONESTA (vale pros 3 blocos): NADA disso liga trade com capital sozinho. A execução do Motor 2 e os filtros de EV vêm DESLIGADOS por padrão — só logam. O caminho pra mainnet continua o mesmo: rodar em DRY_RUN, encher o ledger, calibrar com dado real, e só então capital pequeno. Validação deste merge: typecheck 13/13, ~404 testes TS verdes, contratos intactos (78/79).

6. Camada de aprendizado — o cérebro do bot

Toda decisão do ZEUS é gravada estruturada pra futuro treinamento de IA e análise. Mesmo perdendo uma oportunidade, ele aprende com o erro. Esta camada é o diferencial estratégico de longo prazo.

Componente	O que coleta / faz
Failure Analytics (60+ campos por falha)	Pra cada operação que perde: quem ganhou, qual gás pagou, qual protocolo, qual hora, qual builder
Competitor Fingerprinting	Perfis ricos dos competidores: hábitos de gás, horários, protocolo favorito, grupos coordenados
PnL Reconciliation	Pra cada operação confirmada: lucro esperado vs lucro real, decomposição da diferença em 6 causas
Finality & Reorg Protection	Detecta quando a blockchain 'volta atrás' (reorg) e cancela operações afetadas
Historical Intelligence (DuckDB)	Base de dados time-series com todos os eventos pra análise + treinamento futuro de IA
Health Monitoring	Servidor HTTP de saúde (/healthz, /readyz, /metrics) pra Fly.io/UptimeRobot detectarem se bot caiu
Observability (Prometheus + Grafana)	22 métricas exportadas + 2 dashboards visuais

■ **Em linguagem simples:** Imagine que o bot tem uma 'caixa-preta' tipo a do avião, mas que coleta **MUITO** mais detalhes. Toda decisão, lucro, perda, competidor visto, gás pago, hora do dia — tudo fica gravado. Em 6 meses isso vira a base pra treinar uma IA que decide melhor que regras fixas.

7. Relatórios automáticos no Discord

O bot envia 3 relatórios automáticos pro Discord (basta configurar a URL do webhook):

Relatório	Quando	O que mostra
PnL Daily	Diário 12h UTC	Lucro/perda do dia, top causas, melhor/pior protocolo, sugestões automáticas
Competitor Weekly	Segunda 14h UTC	Top competidores que nos venceram, gás médio, protocolo favorito de cada um
Failure Weekly ■	Segunda 15h UTC	Falhas da semana agrupadas por categoria, oportunidades perdidas, padrões temporais

■ **Em linguagem simples:** É como ter um gerente que te manda 3 relatórios automáticos: um do dia, um da semana sobre concorrentes, e um da semana sobre o que deu errado. Você lê em 5 minutos e sabe exatamente onde ajustar.

8. O que ainda falta para o primeiro lucro em mainnet

8.1 Grupo A — Operacional

Bloqueador	Tempo / Custo	Decisão pendente
Multisig Safe Wallet	2-3h · ~R\$ 50 gas	Threshold (2-of-3?) + 2 hardware wallets
Capital inicial Phase 7	Imediato	Quanto? Sugestão: R\$ 2.500-15.000 (US\$ 500-3.000)
2 semanas DRY_RUN mainnet	14 dias · ~R\$ 50 Fly.io	Onde hospedar (Fly.io)
Calibração MAX_SLIPPA GE_BPS	Auto (DRY_RUN gera)	—
Tenderly alerts	1-2h · grátis	Quais alertas ativar
Deploy contratos Base mainnet	Técnico · ~horas	Apertar o botão — destrava o resto

■ **Em linguagem simples:** Estes são bloqueadores de NEGÓCIO e operação, não de estratégia (o edge já foi decidido na Fase 4c: nicho sub-servido). É a parte que depende de você decidir (capital) e configurar (Safe wallet, Tenderly, hospedagem) + o deploy técnico em mainnet. Sem isso, o ZEUS está pronto pra rodar mas não pode sair do teste pra produção.

8.2 Grupo C — Volume (expansão de mercado)

Sprint	Status	Borrowers a mais
Sprint 1: Seamless (Aave fork Base)	Não feito	+200
Sprint 2: Multi-chain (Arb + OP mainnet)	Não feito	+1.500
Sprint 3: Compound + Morpho + Moonwell	Compound feito, Morpho parcial	+5.200
Total projetado		123 → 7.000 borrowers

■ *Em linguagem simples:* Hoje o ZEUS olha 123 'casas em apuros' pra liquidar em Base. Com a expansão, vai olhar 7.000. Mesmo bot perfeito tem probabilidade baixa de pegar oportunidade com universo tão pequeno — volume é pré-condição matemática.

8.3 Motor 2 — Ponte do radar (MIS) até a execução (LEMBRETE)

O MIS (seção 5.3) hoje é um RADAR em observação pura: varre, identifica, estima e ranqueia — e para no log. Ele NÃO está ligado ao executor de flashloan (ZeusArbExecutor), e isso é uma escolha deliberada, não um esquecimento. Ligar a execução do Motor 2 exige as peças abaixo, e só faz sentido DEPOIS de coletar dias de persistência e confirmar que existe par com ineficiência real que aguenta nosso notional.

Peça pra ligar MIS → executor	Status
txBuilder de rota de arb (buy leg + sell leg) → executeFlashloanArbitrage	Não feito
simulateArbitrage (eth_call atômico) antes de qualquer submit	Reusa o do liquidator — não fiado ao MIS
Allowlist de token do arb (fee-on-transfer / honeypot passam pela sim e quebram na execução)	Existe (tokenSafety) — não fiada ao MIS
Sizing do notional pela liquidez do pool (em vez de \$10k fixo)	FEITO (optimizeFlashLoan) — acha pico de lucro + teto viável
Wire da caixa-preta (scorer/reconciler venue='arb')	Pendente — item 5 do plano do Motor 2
Gates herdados (kill switch / cooldown / gas / slippage floor)	Existem no liquidator — falta plugar no caminho de arb

■ *Em linguagem simples:* Pensa no MIS como o radar de um navio de guerra: ele aponta onde estão os alvos e diz se valem o tiro. Mas o gatilho (executor) é um sistema separado, ligado de propósito só quando o comandante decide. Radar bom não atira sozinho — primeiro a gente confirma que o alvo é real (persistência) e que o canhão alcança (sizing/segurança).

9. Funcionalidades pré-prontas pra ativar depois

Estes itens do checklist 16 estão mapeados, decompostos e priorizados — mas não foram implementados ainda porque dependem de coisas externas (dinheiro, dados reais de mainnet, ou mais tempo de desenvolvimento).

9.1 Bloqueado por capital externo (\$199/mês Alchemy ou similar)

Item	O que vai fazer
Item 2: Mempool Classification	Ver transações antes delas serem confirmadas (vantagem de microssegundos)
Item 3: State Simulation Local	Simular operações 20-30x mais rápido em uma cópia local da blockchain

■ **Em linguagem simples:** É como ter visão raio-X do trânsito (Mempool) e um simulador de direção offline (State Sim). Não dá pra simular sem ver — por isso os dois andam juntos. Custo: ~US\$ 199/mês cada serviço.

9.2 Bloqueado por dados reais de mainnet pra calibrar

Item	Por que precisa de dados reais
Item 6: Dynamic Gas Strategy	Pra calibrar gorjeta ótima, precisa baseline real
Item 11: Priority Queue	Pra rankear oportunidades, precisa volume real
Item 13: Shadow Mode (A/B testing)	Precisa 'produção' rodando ao lado pra comparar
Item 14: Opportunity Scoring	Pesos calibrados contra histórico real

■ **Em linguagem simples:** Implementar essas peças SEM dados reais é teatro de engenharia — você calibra contra hipótese, não contra realidade. Faz sentido implementar quando tiver 2 semanas de DRY_RUN mainnet com dados de slippage real.

9.3 Podem rodar AGORA sem mainnet (só esforço)

Item	Esforço
Item 1: Latency Tracking (p50/p95 por etapa)	3-4h
Item 7: Multi-Broadcast no liquidator	22h
Item 8: Bundle Strategy (multi-tx atomic)	22h
Item 16A: AI base infra (ONNX Runtime)	20h

■ **Em linguagem simples:** Estes 4 itens não dependem de dinheiro nem de mainnet — só de horas. Mas a recomendação é PAUSAR e validar o motor base primeiro com Phase 7 live. Sem saber se o motor liga, não vale otimizar o exaustor.

10. Caminho até o primeiro lucro em mainnet

Quando	O que acontece
HOJE	Decisão: capital inicial + qual edge perseguir
+ 1 dia (2-3h)	Criar Safe Wallet 2-of-3 + hardware wallets
+ 2 dias (2h)	Setup Tenderly alerts (6-8 essenciais)
+ 3 dias	Deploy DRY_RUN na Fly.io (Base mainnet)
+ 17 dias (14d)	Operar DRY_RUN observando, coletando calibração
+ 18 dias	Review do calibration log → go/no-go Phase 7
+ 20 dias	Phase 7 LIVE com capital pequeno (US\$ 500-3.000)
+ 6 semanas	Review primeiro lucro / métricas / bugs
+ 8 semanas	Audit externo se TVL > US\$ 10k

■ **Em linguagem simples:** Em 3 semanas o ZEUS pode estar operando capital real. Depois disso, é validar e escalar. Nada nesse caminho depende de mais código — é tudo decisão + setup + observação.

11. Doutrina estratégica — como o ZEUS compete

ZEUS não compete por infra — compete por **posicionamento**. Esta doutrina foi decidida em 2026-05-27 e está salva como referência permanente do projeto. Cada decisão futura deve passar por ela.

11.1 Pilha de edges (6 níveis mutualmente reforçados)

Edge	O que é
1. Mercados sub-servidos	Atacar Morpho/Moonwell/Seamless — menos bots, menos saturação, win rate sobe muito
2. Long-tail collateral	Especialização em colaterais que mainstream ignora (não só ETH/WBTC/USDC)
3. Backrun seletivo	Filtros contextuais (swap size, pool, hora) — NÃO backrun genérico
4. Edge temporal	Identificar horários onde competidores somem, builders mais lentos
5. Competitor-aware execution	Adaptar estratégia ao competidor visto (gas, relay, bribe)
6. Intelligence moat	Dataset proprietário acumula com tempo — moat definitivo de longo prazo

■ **Em linguagem simples:** Como uma loja de bairro contra um supermercado: a loja não tem mais infra nem mais capital, mas conhece o cliente pelo nome (Edge 5), sabe quando concorrentes fecham (Edge 4), tem produto exótico (Edge 2), e está num bairro que a rede grande não considera atender (Edge 1). Cada ano, conhece mais (Edge 6). Não compete pelo mesmo cliente — captura outro mercado.

11.2 Princípio de expansão (CRÍTICO)

Não expandir por chain. Expandir por oportunidade estrutural.

Talvez no futuro: Arbitrum seja melhor pra liquidation, Base melhor pra backrun, Avalanche melhor pra arbitragem, Optimism melhor pra low competition. ZEUS vira **multi-specialized execution engine** — cada chain com especialização escolhida por dado, não por hype.

■ **Em linguagem simples:** É como uma empresa de logística: não abre filial em todas as cidades. Abre onde os dados mostram que vale. E cada filial se especializa no que aquela cidade pede.

11.3 Plano de validação multi-chain (3 chains paralelas)

Mitigação do risco 'focar em Base sem dar lucro': rodar DRY_RUN simultâneo em **Base + Optimism + Arbitrum** por 14 dias. Mesmo bot, mesma config, 3 deployments paralelos. Custo: ~US\$ 20-30/mês Fly.io extra. Output: matriz com win rate hipotético / lucro estimado / competição por combo. Capital real (Phase 7 live) concentra na vencedora.

■ **Em linguagem simples:** É a diferença entre 'vou abrir loja em Manaus' (aposta cega) e 'vou colocar 3 stands de pesquisa em 3 cidades por 2 semanas e abrir a loja onde teve mais demanda'. Stand custa pouco; loja errada custa caro.

11.4 Chain Profitability Score (implementado nesta sessão)

Toda decisão de 'onde focar capital' passa por uma fórmula objetiva. Não chute, não narrativa, não FOMO. **Ciência.** O ZEUS calcula este score automaticamente pra cada combinação (chain × protocolo) durante o DRY_RUN.

Fórmula:

Componente	Peso	Fonte de dado
Opportunity Density	+0.25	intelligenceStore (DuckDB) — ops vistas por hora
Expected Win Rate	+0.30	pnlReconciler — confirmed / total
Net Profitability	+0.30	pnlReconciler — lucro médio USD por op
Competition Intensity	−0.15	senderRegistry — competidores únicos vistos
Score final	[0, 1]	Mais alto = melhor pra concentrar capital

■ **Em linguagem simples:** Pense em um sistema de notas escolares: cada matéria tem peso diferente, e a nota final é uma média ponderada. O ZEUS dá 'nota' pra cada combinação de chain × protocolo, e o capital real vai pra que tira a maior nota. É decisão científica, não emocional.

11.5 Exemplo de ranking gerado

#	Chain × Protocol	Score	Ops/h	Win%	\$/op	Competidores
■	Base × Morpho	0.78	8.2	65%	\$42	8
■	Base × Moonwell	0.71	5.5	55%	\$38	12
■	Optimism × Seamless	0.62	4.1	48%	\$35	15
4	Base × Compound III	0.45	12.0	25%	\$28	32
5	Base × Aave V3	0.38	15.5	18%	\$25	45

■ **Em linguagem simples:** Neste exemplo hipotético: apesar de Aave V3 ter MAIS oportunidades por hora (15 vs 8 do Morpho), o score do Morpho é o dobro — porque ganhamos 65% lá vs 18% no Aave. Volume bruto sem win rate é prejuízo. Esta tabela é o que o ZEUS gera automaticamente no Discord semanal após o DRY_RUN.

11.6 Decisões anti-tese (o que NÃO fazer)

Tentação	Por que evitar
Competir no Aave V3 mainstream com mempool premium pago	Capital advantage não é nosso edge — top 5 liquidators sempre ganham lá
Backrun genérico (todo swap > \$X)	Edge 3 é seletivo. Genérico = perda garantida
Expandir pra Ethereum mainnet pra ter volume	Competição máxima — saímos da nossa zona de edge
Pagar bribe agressivo no Aave V3 pra forçar win	Queima capital sem construir moat

■ **Em linguagem simples:** Regra geral: decisão que tira o ZEUS de 'competição imperfeita' e coloca em 'guerra de infra' é decisão errada. Sempre.

12. Potencial de lucro — estimativas honestas

Disclaimer importante: ZEUS ainda não rodou em mainnet com capital real, então tudo aqui é projeção baseada em benchmarks públicos de outros liquidators Base/Ethereum + premissas do nosso setup atual. Cada cenário tem racional explícito pra você poder ajustar se discordar das premissas.

■ **Em linguagem simples:** Pense nisso como o cardápio de um restaurante novo: o dono sabe o custo dos pratos e o preço médio do mercado, mas só vai saber o faturamento real depois de 1 mês operando. As estimativas abaixo são 'pratos do dia' projetados — não promessas.

12.1 Premissas do cálculo

Variável	Valor assumido
Capital inicial (gas reserve only)	US\$ 500-3.000
Lucro vai pra capital próprio reinvestido	45% (princípio salvo em memory)
Bonus médio Aave V3 / Compound III	5-10% do debt (varia por collateral)
Bonus médio Morpho Blue	5-12% (mercados isolados pagam mais)
Custo médio de gas por tx em Base	US\$ 0.05-0.30
Custo médio de bribe (se ativo)	20-40% do profit bruto
Slippage médio típico em pools profundos	30-80 bps
Borrowers cobertos hoje (Base Aave V3)	123
Borrowers cobertos após Sprint 1+2+3	~7.000
Volume médio mensal liquidations Base Aave V3	US\$ 500k-2M (DefiLlama)

12.2 Win rate (taxa de vitória) — fator crítico

Win rate é a % das vezes que o ZEUS ganha a oportunidade contra os outros bots. É o fator que mais influencia o lucro total. Benchmarks observados em Base:

Cenário	Win rate típico	Quem está nesse range
Pessimista (sem edge, sem mempool premium)	5-15%	Bot novo competindo no top-tier Aave V3
Realista (niche advantage Morpho/Moonwell)	20-35%	Bot focado em mercados menos competidos
Otimista (mempool premium + estratégia exclusiva)	40-60%	Top 5 liquidators estabelecidos

■ **Em linguagem simples:** É como pesca: num lago cheio de pescadores experientes (Aave V3 mainstream), você pega 1 peixe a cada 10 que vê. Num lago menor com menos gente (Morpho, Moonwell), você pega 3 a cada 10. Pra pegar 5 a cada 10 precisa ter equipamento melhor que todo mundo — isso é mempool premium + estratégia exclusiva.

12.3 Cenário pessimista (1º mês operando)

Métrica	Valor estimado
Win rate assumido	10%
Oportunidades vistas/mês (123 borrowers Base)	~30-50
Operações vencidas	3-5
Lucro bruto por operação (média)	US\$ 15-40
Lucro bruto total mês	US\$ 45-200
Custo gas + bribe	US\$ 30-100
LUCRO LÍQUIDO PROJETADO	US\$ 15-100
Suficiente pra cobrir Fly.io + RPC?	Sim, marginalmente

■ **Em linguagem simples:** No pior cenário, o bot SE PAGA mas não enriquece. É o cenário esperado se entrarmos no Aave V3 mainstream sem nenhum diferencial competitivo. Útil pra validar que tudo funciona, mas não é cenário de scale.

12.4 Cenário realista (3-6 meses, com niche advantage)

Métrica	Valor estimado
Win rate assumido	25%
Cobertura expandida (Sprint 1+3 — Seamless + Compound + Moonwell)	~3.000 borrowers
Oportunidades vistas/mês	~300-600
Operações vencidas	75-150
Lucro bruto por operação (média)	US\$ 20-60
Lucro bruto total mês	US\$ 1.500-9.000
Custo gas + bribe	US\$ 500-3.000
LUCRO LÍQUIDO PROJETADO	US\$ 1.000-6.000/mês
Anualizado (extrapolação)	US\$ 12.000-72.000/ano

■ **Em linguagem simples:** Esse é o cenário-alvo: 'lago menos pescado' com niche advantage em Morpho/Moonwell/Seamless. Em 3-6 meses, com calibração contínua e dataset crescendo, o bot pode passar do break-even pra geração consistente de R\$ 5-30 mil/mês líquido. Premissa: edge funcionando + 0 incidentes graves de segurança.

12.5 Cenário otimista (6-12 meses, com mempool premium + expansão completa)

Métrica	Valor estimado
Win rate assumido	40%
Cobertura completa (todos 3 sprints + multi-chain)	~7.000 borrowers
Mempool premium ativo (Alchemy Growth+)	Sim — US\$ 199/mês
Oportunidades vistas/mês	~700-1.500
Operações vencidas	280-600
Lucro bruto por operação (média, com motor 3 backrun ativo)	US\$ 30-100
Lucro bruto total mês	US\$ 8.000-60.000
Custo gas + bribe + infra	US\$ 3.000-15.000
LUCRO LÍQUIDO PROJETADO	US\$ 5.000-45.000/mês
Anualizado (extrapolação)	US\$ 60.000-540.000/ano

■ **Em linguagem simples:** Cenário onde o ZEUS vira operação séria. Requer: 6+ meses de dados pra calibrar IA (Item 16A), mempool premium pago, expansão completa de chains/protocolos, audit externo, multisig robusto. Não é fantasia — é o que top 5 liquidators Base fazem hoje.

12.6 Lucro por motor (cenário realista, mês típico)

Motor	Contribuição estimada	Quando ativa
Motor 1: Liquidations	60-70% do lucro	Sempre (mercado normal + crashes)
Motor 2: Cross-DEX Arb	10-20% do lucro	Volume alto, sem necessidade de mempool
Motor 3: Backrun	20-30% do lucro	Volatilidade — depende mempool premium

■ **Em linguagem simples:** O Motor 1 (liquidations) é o 'pão com manteiga' — funciona em qualquer mercado e responde pela maior parte do lucro. Os outros 2 são 'bônus' que aparecem em regimes específicos. A descorrelação garante que o bot ganha em qualquer cenário.

12.7 Fatores que aumentam o lucro

Fator	Impacto
Cobertura mais protocolos (Seamless, Moonwell, Morpho)	+200% a +500% de oportunidades
Multi-chain expansion (Arb + OP + Polygon + Avalanche)	+100% a +300% por chain
Mempool premium (Alchemy / Blocknative)	+50% a +200% de win rate
Edge específico (niche advantage)	+30% a +100% de win rate
IA treinada com 6 meses de dataset	+10% a +30% de margem
Audit externo (mais confiança = mais capital)	Permite escalar bribe agressivo

12.8 Riscos que reduzem ou zeram o lucro

Risco	Mitigação atual
Bug em contrato → fundos drenados	7 fixes auditados (B-1 a B-7) + audit externo planejado
Chave executor comprometida	Multisig Safe Wallet (Phase 7)
Reorg cancelando ops confirmadas	FinalityTracker + OrphanRecoveryManager
Oracle manipulado / stale price	ChainlinkStalenessChecker (Grupo B)
Protocolo pausa governança no meio da tx	PauseDetector (Grupo B)
RPC fica lento / down	Multi-broadcast planejado (Item 7)
Volume cai 80% em bear market longo	3 motores descorrelacionados — pelo menos 1 ativa
Bot perde 5 ops seguidas (estratégia desatualizada)	FailureTracker cooldown automático
Daily loss > limite configurado	PnL kill switch automático
Competidor com edge superior (latência, capital)	Niche advantage estratégico em mercados menos saturados

■ **Em linguagem simples:** Cada risco listado tem uma proteção ativa correspondente. Não significa que ZEUS é imune — significa que está coberto pra cenários conhecidos. O risco real é o cenário DESCONHECIDO — por isso 2 semanas DRY_RUN antes de capital real é inegociável.

12.9 Resumo financeiro consolidado

Horizonte	Cenário pessimista	Cenário realista	Cenário otimista
Mês 1 (validação)	US\$ 15-100	US\$ 200-800	US\$ 1.000-3.000
Mês 3 (com expansão)	US\$ 50-300	US\$ 1.500-6.000	US\$ 5.000-15.000
Mês 6 (com IA inicial)	US\$ 100-500	US\$ 3.000-10.000	US\$ 10.000-30.000

Mês 12 (cenário completo)	US\$ 200-1.000	US\$ 5.000-15.000	US\$ 20.000-45.000
---------------------------	----------------	-------------------	--------------------

■ **Em linguagem simples:** Pra contextualizar em real (assumindo US\$ 1 = R\$ 5): o cenário realista no mês 6 representa R\$ 15-50 mil/mês líquido. O cenário otimista no mês 12 representa R\$ 100-225 mil/mês. Mas LEMBRE: tudo isso é estimativa — só 14 dias de DRY_RUN real vão te dizer onde você cai na curva.

13. Investimento de infraestrutura (RPC, mempool, hosting)

O ZEUS depende de provedores externos pra falar com a blockchain (RPC), ver transações pendentes (mempool) e ficar online 24/7 (hosting). Esta seção lista cada custo e QUANDO ele se torna necessário — pra não pagar por coisa que ainda não usamos.

■ **Em linguagem simples:** Pense nisso como as contas fixas de um restaurante: aluguel (hosting), conta de luz (RPC) e o fornecedor premium de ingredientes raros (mempool). Você liga a luz desde o dia 1, mas só contrata o fornecedor premium quando o prato que precisa dele entra no cardápio.

13.1 Provedores e função de cada um

Provedor	Função no ZEUS	Necessário quando
dRPC	RPC reads pesados (discovery on-chain: getLogs, multicall, posições). Agregador com fallback embutido.	Já — fase DRY_RUN
Alchemy	RPC + WSS + MEMPOOL (pending txs pro backrun). Cobre tudo numa conta, mas mempool é o diferencial premium.	Mempool: só quando ativar backrun (motor 3)
Fly.io	Hosting 24/7 do bot (1 instância por chain).	Já — deploy DRY_RUN
Tenderly	Alertas on-chain (tx revertida, owner mudou, saldo baixo).	Antes do Phase 7 live
The Graph	Subgraph Aave V3 core (discovery). Forks usam on-chain (sem custo).	Já — coberto por API key existente

13.2 Custo estimado por provedor

Provedor	Plano	Custo estimado/mês	Observação
dRPC	Pay-as-you-go / paid	US\$ 50-100	Volume de reads dos 4 protocolos + cache acumulativo. Free tier NÃO aguenta.
Alchemy	Growth	US\$ 49	RPC + WSS. Sem mempool premium ainda.
Alchemy	Growth+ (mempool)	US\$ 199	Inclui alchemy_pendingTransactions (pro backrun). Só quando ativar motor 3.
Fly.io	Hobby/Launch	US\$ 5-30	1 instância por chain. 3 chains DRY_RUN ≈ US\$ 15-30.
Tenderly	Free / Pro	US\$ 0-50	Free tier cobre ~30 alerts. Pro se escalar.

■ **Valores são estimativas** (conhecimento até jan/2026). Confirmar no painel de cada provedor antes de assinar — pricing muda, e verificar especificamente: (1) alchemy_pendingTransactions disponível em Base no tier escolhido; (2) limite de CU/requests do plano aguenta nosso volume de reads.

13.3 Faseamento do investimento

Estratégia: dRPC pros reads (barato, resiliente) + Alchemy pro mempool (premium) — divisão de carga, não redundância. Mas o mempool só entra quando ativarmos o backrun. Liquidations (motor principal) NÃO precisam de mempool.

Fase	O que assinar	Custo total/mês
------	---------------	-----------------

Fase atual (DRY_RUN + Phase 7 liquidations-first)	dRPC paid OU Alchemy Growth + Fly.io + Tenderly free	US\$ 60-130
Fase backrun (ativar motor 3 + mempool)	dRPC (reads) + Alchemy Growth+ (mempool) + Fly.io + Tenderly	US\$ 260-330
Fase escala (multi-chain + volume alto)	dRPC + Alchemy Scale + Fly.io (N instâncias) + Tenderly Pro	US\$ 400-600

■ **Em linguagem simples:** Decisão registrada: Alchemy pago cobre RPC + mempool numa conta só, mas usar Alchemy pra TUDO (incluindo reads pesados) queima compute units rápido. Por isso a divisão: dRPC carrega os reads (barato), Alchemy reservado pro mempool (premium). Assinar os dois só vale a partir da fase backrun — agora, 1 provedor robusto basta.

13.4 Por que o custo de RPC subiu

Com a expansão pra 4-5 protocolos (Aave + Seamless + Compound + Morpho + Moonwell), o discovery on-chain ficou RPC-intensivo: cada ciclo faz getLogs + multicall pra cada protocolo, e a lista de devedores monitorados (cache acumulativo) cresce com o tempo. Isso é o preço da cobertura ampla (123 → ~7.000 borrowers) — e justifica o RPC pago desde a fase DRY_RUN.

14. Resumo executivo final

O que está pronto

- Camada de observação completa (failure analytics, competitor fingerprinting, finality, PnL reconciliation, health server, intelligence DuckDB, Prometheus + Grafana)
- Todos os 6 gaps técnicos do Grupo B fechados (oracle staleness, pause detection, gas cache, multi-collateral, partial optimization, multi-hop swaps)
- Multi-chain ready — Polygon e Avalanche entram com 1 arquivo de config cada
- 194 testes verdes, 9 workspaces typecheck verdes, 53 testes de contrato verdes
- 3 relatórios Discord automáticos prontos (PnL daily, Competitor weekly, Failure weekly)
- 2 dashboards Grafana prontos pra importar

O que falta

- Decisão de capital inicial Phase 7 (US\$ 500-3.000)
- Decisão de edge competitivo (sugestão: niche advantage Morpho/Moonwell)
- Setup Safe Wallet multisig 2-of-3
- Setup Tenderly alerts (6-8)
- Hospedagem Fly.io + 14 dias DRY_RUN mainnet pra calibrar
- Expansão de protocolos pra ter volume estatístico (123 → 7.000 borrowers)

Conclusão honesta

O ZEUS tem TODA a infraestrutura de aprendizado e captura técnica pronta. O próximo passo de maior impacto NÃO é mais código — é decisão de capital, decisão de edge, e 14 dias de DRY_RUN mainnet pra calibrar. Em 3 semanas pode estar operando capital real.

15. Primeiro Voo Instrumentado — o plano de validação na mainnet

O primeiro voo NÃO é pra lucrar — é pra a caixa-preta responder, com dado real e risco mínimo, se existe dinheiro no nosso nicho. Estreito, instrumentado, flashloan-only. A ideia: em vez de adivinhar (eu ou você), deixar os DADOS decidirem se a tese vive.

■ *Em linguagem simples:* Temos um carro de corrida com motor testado no dinamômetro (fork tests) — mas que nunca saiu da garagem. Este plano é a primeira volta na pista real: devagar, com todos os sensores ligados, medindo tudo. A caixa-preta do ZEUS é o que torna isso seguro — cada volta (boa ou ruim) vira dado.

15.1 Escopo — deliberadamente estreito

Dentro do voo	Fora (por enquanto, de propósito)
1 chain: Base	Polygon / Avalanche
Motor 1: só Aave V3 + Compound III (os mais validados)	Moonwell (sem fork test) · Morpho · Seamless
Motor 2 (MIS): observação pura em paralelo (zero risco)	Motor 3 (precisa mempool) · ponte MIS→executor

■ *Em linguagem simples:* Por que estreito: a caixa-preta gera sinal LIMPO com 1-2 frentes. Com 5 motores/chains ao mesmo tempo, vira ruído e a gente não sabe o que funcionou.

15.2 As 3 fases

Fase	O que	Risco
0 — DRY_RUN mainnet (dias 1-3)	Deploy + liquidator em modo observação. Valida os INSTRUMENTOS da caixa-preta contra fluxo real (o cockpit nunca rodou ao vivo)	ZERO
1 — Armado, cap mínimo (sem 1-2)	Flip pra mainnet, MAX_TRADE ~US\$ 200-500. Flashloan-only + atômico → downside é só gas	Baixo
2 — Decisão (sem 2-4)	Lê as 4 métricas → tese vive, morre ou pivota	—

15.3 As 4 métricas que decidem (vive ou morre)

Métrica	Vive se...	Morre/pivota se...
1. Densidade (oport./semana)	> punhado/semana	≈ 0 → nicho seco; ir pros sub-servidos
2. Win rate (das tentadas, % ganhas)	> 0 e subindo	≈ 0% → gap de velocidade fatal aqui
3. Net real por captura (pós gas)	positivo	negativo consistente → não fecha
4. MIS: pares persistentes no gate	≥ 1 par	0 pares → tese de arb falsificada (por ora)

■ *Em linguagem simples:* O melhor cenário do plano NÃO é 'lucrar' — é **DESCOBRIR** a verdade barato. Se ambos os motores derem seco em 4 semanas, a tese precisa mudar — e a gente descobriu isso com ~US\$ 0 de risco, em vez de gastar meses construindo o 6º protocolo pra um mercado que não

paga. Detalhe operacional em docs/FIRST_FLIGHT.md.

16. Motor 3 — refit pra realidade da Base em 2026

Pesquisa web (2026-05-29) revelou que o conceito original do Motor 3 (backrun via bundle atômico privado) está PARCIALMENTE inválido pra Base. A boa notícia: a versão refit pode sair MAIS BARATA do que pensávamos.

16.1 O que mudou no mercado (e nos pegou desatualizados)

Provedor	Status atual	Impacto no ZEUS
Blocknative	Cessou operações jun/2025 (equipe → Deloitte)	blocknativeRelay.ts virou letra morta
Atlas (FastLane)	Chainlink comprou jan/2026 — agora exclusivo Chainlink SVR	atlasRelay.ts virou letra morta
Eden Network	Fechou oficialmente	Nem entrava no nosso código
Flashbots	Vivo no L1. Na Base construiu o Flashblocks (op-rbuilder) — outro modelo	Endpoint relay clássico NÃO se aplica à Base
bloXroute	Vivo, suporta Base (streams + submission). Bundle atômico em ETH/BSC (Base a confirmar)	Candidato pago

■ **Em linguagem simples:** É a evolução natural do mercado MEV: a era dos relays L1-style (2021-2024) deu lugar ao modelo L2 com sequenciador centralizado + preconfirmações rápidas. A Base é o caso emblemático.

16.2 A realidade técnica da Base — e por que muda o Motor 3

A Base NÃO TEM mempool público — o sequencer da Coinbase não faz gossip. Único caminho de entrada é via RPC. Em julho/2025 a Base lançou os Flashblocks (pré-blocos a cada 200ms) via op-rbuilder, construído pela Flashbots. O jogo de MEV ali é leilão de PRIORITY FEE dentro da janela de 200ms — NÃO é bundle atômico Flashbots-style.

■ **Em linguagem simples:** Em uma frase: a premissa antiga ('compro Alchemy mempool US\$ 199/mês pra ver a baleia') era DE Ethereum L1. Na Base não há mempool pra assinar — o que há é o feed de Flashblocks (que é OUTRO produto). Isso muda a conta de infra do Motor 3 pra baixo.

16.3 Motor 3 REFIT — "Flashblocks-Priority Backrun"

Passo	Como funciona
1. Ver a baleia	Assina WebSocket de Flashblocks (pré-blocos 200ms) via provider que suporte
2. Classificar	Detecta swap grande no pré-bloco que cria dislocação explorável
3. Montar arb	Cross-DEX que captura a dislocação (mesmo motor do MIS)
4. Submeter	Via RPC com priority fee competitivo pra entrar no pré-bloco N ou N+1
5. Atomicidade	Flashloan + minProfitWei no contrato: ou lucra, ou reverte (só perde gas)

■ **Em linguagem simples:** Trade-off honesto: sem bundle atômico, em alguns cenários a tx da baleia pode sumir/reverter e a nossa fica desencaixada. Mas o flashloan + min profit no contrato garantem o pior caso = só gas perdido (zero risco de capital). O edge passa a ser LATÊNCIA + PRICING DE GAS,

não bundle.

16.4 Implicação de custo (boa notícia)

	Antes (estimado)	Refit (provável)
Mempool feed	Alchemy Growth+ US\$ 199/mês	Flashblocks WS (incluído em Alchemy free? Chainstack? a confirmar)
Relay pago	Blocknative/Atlas (mortos)	Nenhum — priority fee auction direto no sequencer
Hosting	Comum	Low-latency próximo ao sequencer (US-East no Fly.io)
Custo extra/mês	~US\$ 199	Provavelmente próximo de zero além do já planejado

■ **Em linguagem simples:** Conclusão: o Motor 3 nessa formatação **NÃO** precisa de novo investimento de infra significativo. Mas **EXIGE** pesquisa de fechamento antes de codar: (1) qual provider serve Flashblocks WS de verdade, (2) atomic bundle no bloXroute Base é viável e a que preço, (3) Coinbase Searcher Program existe? Detalhe técnico em docs/MOTOR3_REFIT.md.

Princípio que sobreviveu: o flashloan atômico do contrato continua sendo a peça que torna isso seguro. O Motor 3 refit muda COMO a tx entra no bloco, não muda o que o contrato faz.

Este relatório foi gerado automaticamente do estado real do código em 2026-06-22. Typecheck 13/13 · execution-utils 256/256 · mis-scanner 6/6 · liquidator 22/22 · contratos 67 unit + 34 fork · Branch: main · Motor 2 (radar MIS) + Polygon/Avalanche code-ready · Motor 3 refit pesquisado.